

SYDENCE

Studio Operating System

Comprehensive Project Brief

From Concept to Production

A PegBit Studio Product · Powered by Omos

v2.0 · May 2026 · Production Ready

Confidential — Internal Use Only

00 Executive Summary

Sydence is the operating system for small creative and product studios. It replaces five or more disconnected tools with one opinionated workspace — combining project management, AI knowledge, team development, and organisational learning in a single product.

Built first for PegBit Studio itself, Sydence grew from a single-tenant internal tool into a full multi-tenant SaaS platform. Every feature exists because it was needed in production — not because a competitor had it. This brief documents everything: what was built, how it was built, and where it goes next.

Dimension	Status
Product Status	Production Ready (v2.0)
Launch Stage	Pilot — targeting 5 studios in 10 weeks
Multi-tenant	Full isolation per studio via Supabase RLS
AI Integration	Omos: OpenAI GPT-4 + 1536-dim vector embeddings
Core Pages	7 (Dashboard, Board, Pipeline, Ideas, Standups, Team, Colony Lab)
Deployment	Vercel (frontend + API) + Supabase (database)
Security	RLS on every table, role-based access (5 roles)
Built By	PegBit Studio — Lagos, Nigeria

01 Product Identity

The One-Line Definition

SYDENCE

The operating system for small creative and product studios — the first tool that combines project management, AI knowledge, and team development in one opinionated workspace.

What Sydence Is

Sydence is a complete operating layer for studios — teams that simultaneously run their own products, serve clients, develop junior talent, and learn from every experiment. The workflow architecture is fixed and opinionated. What is configurable is the vocabulary: what each stage is called, what tracks are named, what roles are titled.

This is the Linear model applied to studios: opinionated enough to onboard in a day, flexible enough to feel like your own.

What Sydence Is NOT

Mistaken Identity	The Reality
A billing or invoicing tool	Workamajig and Function Point own this. Sydence does not touch revenue tracking.
A generic PM tool	Asana and Monday serve generic teams. Sydence is specifically for studios.
A blank canvas like Notion	Notion lets you build your own system. Sydence IS the system.
A horizontal AI PM tool	Height tried this and shut down in September 2025. Vertical AI PM is the surviving form.
A time tracking tool	Out of scope. Studios use Harvest or Toggl — Sydence integrates, not replaces.

02 The Problem

How a 10-Person Studio Actually Operates Today

The average small creative or product studio runs on a patchwork of five or more tools that were never designed to work together:

Tool	What it does	Monthly cost (10 people)
Notion Plus	Wiki, knowledge, loose PM	\$100
Asana Starter	Task management	\$110
Geekbot / Range	Standup logging	\$25
Google Drive	File storage, document sharing	\$130
Notion AI or similar	Generic AI writing assistant	\$100
TOTAL	Five tools. Zero integration.	~\$465/month

The Invisible Costs

Pain	What it looks like in practice
Tool sprawl	Employees context-switch 30+ times per day. 60% more time spent on coordination than actual work.
Knowledge loss	Policies in Google Drive. Decisions in email. Processes in one person's head. 20% of work time spent searching for info that already exists.
No talent layer	Career development lives in a Notion doc nobody reads. No connection between work shipped and progression earned.
No learning system	R&D experiments happen informally in Slack and are forgotten within a week.
Generic structure	No tool understands the studio workflow. Every studio rebuilds its own system in Notion, then maintains it, then watches it break as the team grows.

MARKET

The agency management software market is \$4.6B in 2025, growing to \$6.5B by 2030. No tool in this space is AI-native, priced for small studios, and built with a studio-specific workflow. That gap is Sydence.

03 Target Users

Primary User — The Studio Lead

The founder, director, or creative head of a small studio. The person who decides what tools the team uses, approves project briefs, reviews quality, and is responsible for the team's development.

What they deal with daily	What they need from a tool
Juggling multiple projects across different clients and stages	One pipeline view showing every project and its stage without switching apps
Never knowing what the team is blocked on until too late	Daily standup log that surfaces blockers without requiring a meeting
Junior team questions creating a bottleneck	An AI assistant trained on the studio's own documents
No visibility on who is growing and who is stuck	A team directory with progression levels, project badges, and weekly goals
Spending \$400+ per month on non-integrated tools	One workspace at \$150/month that replaces all of them

Ideal Customer Profile (ICP)

Dimension	Profile
Studio type	Product studios · Design agencies · Content studios · Creative consultancies
Team size	3–20 people. Too small for Workamajig. Too structured for a blank Notion setup.
Phase 1 Geography	Lagos · Nairobi · Accra · Diaspora studios in UK and US
Phase 2 Geography	Pan-African expansion · Southeast Asia · Latin America
Signal	Currently using Notion + Asana + Slack + Google Drive. Studio Lead frustrated with context switching.

04 What Was Built

Sydence is a complete multi-tenant SaaS platform built on Next.js 14 and Supabase. It comprises seven core pages, an embedded AI assistant (Omos), a full admin layer, and a role-based permission system.

The Seven Core Pages

#	Page	Route	What it does	Access
01	Dashboard	/dashboard	Studio at a glance. Quick stats, recent activity feed, upcoming milestones, team member status.	All authenticated users
02	Board	/board	Kanban project board. Columns: Backlog → This Week → In Progress → Review → Done. Drag-drop cards, colour-coded by track.	All members (viewers: read-only)
03	Pipeline	/pipeline	Full project lifecycle. Six stages: Brief → Approved → In Development → Review → Staging → Deployed. Brief editor and file attachment included.	All members (viewers: read-only)
04	Ideas	/ideas	Idea submission pipeline. Any member submits. Owner/Admin approves or rejects with written feedback. Approved ideas promote to Pipeline.	All members
05	Standups	/standups	Async daily log: Yesterday / Today / Blockers. Slack notification on post. Full team visibility. No meeting required.	All members
06	Team	/team	Directory with progression levels (Peg / Beam / Pillar / Keystone), project badges, current project, and weekly goals.	All authenticated users
07	Colony Lab	/colony-lab	R&D experiment log. Hypothesis → Active → Completed / Parked. The only studio tool that treats learning as a first-class product feature.	All members

04B Omos — The AI Intelligence Layer

Omos is not a chatbot. It is an Organisational Intelligence Platform embedded directly in Sydence — not an add-on, not a separate subscription. Omos captures everything the studio knows, makes it accessible in natural language, and creates a permanent record of every answer given.

ORIGIN

The name comes from the ancient Greek *ὤμος* (shoulder) — the shoulder that carries institutional weight so the team does not have to.

Five Capabilities Inside Omos

Capability	What it does
RAG Chat (Knowledge Q&A)	Answers questions using the studio's own uploaded documents. Every answer cites the source document and section. Never guesses — if the answer is not in the knowledge base, Omos says so.
Live Studio Context	Queries live project data, standups, ideas, and team info to answer operational questions: "What is the team working on this week?" "Which projects are blocked?"
Report Generation	Generates four report types on demand: Project Status, Team Digest, Sprint Retrospective, Stakeholder Deck. Saved to history for recall.
Action Execution	Executes write operations via natural language: "Log my standup", "Move X to In Progress", "Submit idea for Y project". Uses OpenAI function calling to detect intent.
Admin Analytics	Admin-only dashboard: daily questions asked, answers returned, unique users, token usage, and feedback ratings (thumbs up/down).

Knowledge Base System

Step	What happens
1. Upload	Admin uploads PDF, DOCX, or TXT file via Settings → Knowledge Base.
2. Parse	Server extracts raw text using pdf-parse (PDF) or mammoth (DOCX).
3. Chunk	Text is split into meaningful sections (title + content pairs).
4. Embed	Each chunk is sent to OpenAI text-embedding-3-small → 1536-dimension vector.

5. Store	Chunk stored in studio_knowledge table (Supabase pgvector) with studio_id, document_name, section_title, content, embedding.
6. Search	When user asks a question, the question is embedded and cosine similarity search finds the top-k relevant chunks scoped to the studio.
7. Answer	Top chunks are injected as context into the Claude/GPT-4 prompt. Response streams to the user via SSE.
8. Cite	Every answer includes the document name and section as a citation.

05 Technical Architecture

Technology Stack

Layer	Technology	Why
Web Frontend	Next.js 14 (App Router) + TypeScript + Tailwind CSS	PegBit standard stack. All builders already know it. SSR + SSG for performance.
Backend	Next.js API Routes (Node.js on Vercel serverless)	Same repo as frontend. No separate backend to deploy or maintain.
Database	Supabase (PostgreSQL + Auth + Storage + pgvector)	Multi-tenancy via <code>studio_id</code> + RLS. pgvector powers Omos semantic search. Managed backups.
AI / Embeddings	OpenAI API (GPT-4 + text-embedding-3-small, 1536 dims)	Embeddings: 1536 dimensions for high-precision semantic search. Inference: GPT-4 for quality answers.
Streaming	Server-Sent Events (SSE)	Real-time streaming of Omos responses. No polling, no WebSocket overhead.
Deployment	Vercel (auto-deploy on push from GitHub)	Staging branch → staging env. Main branch → production. Preview deployments on every PR.
Auth	Supabase Auth (JWT-based)	Row-level security tokens. All API routes validate the Bearer token on every request.
Notifications	Slack Incoming Webhooks	Standups post to configured Slack channel. Omos weekly digest optionally sent to Slack.
PDF / DOCX parsing	pdf-parse + mammoth	Server-side document extraction before chunking and embedding.

Multi-Tenant Architecture

Every table in the database carries a `studio_id` column. Row Level Security policies on every table ensure that no query — regardless of what application code does — can ever return rows belonging to a different studio. Security is enforced at the database level, not the application level.

Principle	Implementation
Studio scoping	Every table has <code>studio_id</code> (UUID). All reads and writes are scoped.

RLS on every table	Supabase Row Level Security policies on studios, studio_members, os_projects, os_ideas, os_standups, os_omos_messages, studio_knowledge, and every other table.
JWT validation	Every API route calls supabase.auth.getUser(token) before any data operation. No token = immediate 401.
Service role key	Only used server-side in API routes. Never exposed to the client. Used for write operations that require bypassing the JWT user context.
Admin helpers	Two SECURITY DEFINER Postgres functions: is_studio_member(wid) and is_studio_admin(wid). Prevent RLS recursion.

Core Database Tables

Table	Purpose	Key Columns
studios	Studio entity (tenant root)	id, name, slug, owner_id, settings
studio_members	Who belongs to which studio	studio_id, user_id, role (owner/admin/member/viewer/guest)
studio_invites	Invite tokens for onboarding	studio_id, role, token, expires_at, accepted_at
studio_vocabulary	Configurable labels	studio_id, key (e.g. track_1_name), value
studio_knowledge	Omos knowledge base chunks	studio_id, document_name, section_title, content, embedding (vector 1536)
os_projects	Projects (Board + Pipeline)	studio_id, title, track, status, pipeline_stage, owner_id
os_ideas	Idea submissions	studio_id, title, description, submitted_by, status (pending/approved/rejected)
os_standups	Daily standup logs	studio_id, user_id, yesterday, today, blockers, posted_at
os_omos_conversations	Omos chat sessions	studio_id, user_id, title, created_at
os_omos_messages	Individual chat messages	conversation_id, role (user/assistant), content, tokens_in, tokens_out
os_omos_actions	Actions executed by Omos	studio_id, action_type, parameters, result, executed_at
os_omos_reports	Generated reports	studio_id, report_type, content, generated_at
omos_feedback	Thumbs up/down ratings	message_id, user_id, rating (positive/negative)
os_omos_admin_stats	Admin analytics (aggregated)	studio_id, day, questions_asked, unique_users, total_tokens

06 Security & Permissions Model

Five-Tier Role System

Sydence uses a five-role hierarchy. Permissions are additive — each role inherits everything below it and adds capabilities above it.

Role	Who it is	Key Permissions
Owner	Studio founder / billing owner	Everything. Delete studio. Billing. Change all roles.
Admin	Team lead or operations manager	Manage members. Configure studio. Knowledge base. Analytics. Reports. Invites.
Member	Full team member	Create/edit tasks. Submit ideas. Log standups. View all content.
Viewer	Client, contractor, observer	Read-only. No write operations. Cannot access admin features.
Guest	Temporary or external access	Very limited read. Cannot use Omos. Cannot log standups.

Admin-Only Feature Audit

The following features are validated at both the API level (returns 403 for non-admins) and the UI level (navigation items are hidden or buttons are disabled):

Feature	API Enforcement	UI Enforcement
Analytics Dashboard	GET /api/omos/analytics — checks role = owner admin	Navigation item hidden for non-admins
Knowledge Base Upload	POST /api/omos/ingest — checks role = owner admin	Upload button absent for non-admins
Knowledge Base Delete	DELETE /api/omos/documents — checks role = owner admin	Delete button absent for non-admins
Knowledge Base List	GET /api/omos/documents — checks role = owner admin	Settings page inaccessible
Report Generation	/api/omos/reports — checks role = owner admin	Reports section hidden
Studio Settings	/api/settings — checks role = owner admin	Settings sections gated by role

Member Management	/api/settings/members — checks role = owner admin	Member management hidden
Invite Management	/api/settings/invites — checks role = owner admin	Invites page hidden
Weekly Digest	Cron job validates admin role before sending	Digest configuration hidden

NOTE

Cross-tenant isolation is enforced at the database level via RLS — not the application level. Even if application code has a bug, Supabase will not return rows from another studio. This is the primary security guarantee of the multi-tenant architecture.

07 Step-by-Step: How It Was Built

Syndice began as an internal single-tenant tool for PegBit Studio. This section documents the full development journey — from the original internal OS to a production-ready multi-tenant SaaS platform.

Phase 0 · The Origin (PegBit OS)

WHAT HAPPENED: Before Syndice existed, PegBit Studio ran its operations on an internal tool called the PegBit OS — a single-tenant Next.js application connected to a single Supabase database.

The OS had:

- A Kanban board for project tasks
- A pipeline for tracking project stages
- An ideas submission page
- A standup log
- A basic team directory

KEY DECISION: Rather than retrofit the single-tenant codebase into a multi-tenant system, the PegBit OS was forked into a fresh repository. The Syndice repo is a clean rebuild with multi-tenancy as a first-class concern.

Phase 1 · Architecture Design

WHAT HAPPENED: Before writing a single feature, the team locked the core architectural decisions that all future work would depend on.

KEY DECISIONS MADE:

- Multi-tenancy via `studio_id`: Every table gets a `studio_id` column (not a separate schema per tenant). Simple, queryable, and Supabase-native.
- RLS on every table: Row Level Security policies enforced at the database level. Application bugs cannot cause data leaks.
- `workspace_config` JSONB: All vocabulary configuration stored in a single JSONB column on the `studios` table. One update changes all vocabulary without a schema migration.
- Path-based routing: `app.syndice.io/{studio-slug}`. Simpler than subdomain routing in Next.js.
- Projects / Tasks split: The PegBit OS used one table for pipeline and kanban. Syndice splits into projects (pipeline) + tasks (board). Essential for clean queries at scale.

Phase 2 · Database Schema & Migrations

WHAT HAPPENED: The Supabase database was built with a migration-based approach. Each migration is numbered and idempotent — they can be re-run safely.

MIGRATIONS IN ORDER:

1. 001_initial_schema.sql — Core tables: studios, studio_members, os_projects, os_ideas, os_standups
2. 002_omos_tables.sql — All Omos tables: conversations, messages, actions, reports, knowledge, feedback
3. 003_studio_features.sql — Vocabulary, invites, analytics tables
4. 004_fix_omos_analytics_rls.sql — Security fix: created os_omos_admin_stats with proper RLS; replaced SECURITY DEFINER view with SECURITY INVOKER

Each migration was applied to Supabase via the SQL editor and verified before proceeding to the next.

Phase 3 · Authentication & Studio Creation

WHAT HAPPENED: Studio creation and member invite flows were built. These are the two most critical flows in a multi-tenant SaaS — getting them wrong causes data to be owned by the wrong studio.

STUDIO CREATION FLOW:

5. User signs up via Supabase Auth (email + password)
6. On first login, a studio is auto-created with the user as owner
7. Studio slug is generated from the studio name (lowercase, alphanumeric, hyphens only)
8. User is added to `studio_members` with role = owner
9. All subsequent requests carry the `studio_id` derived from the user's membership

INVITE FLOW:

10. Admin creates an invite for a specific role (admin/member/viewer/guest)
11. A token is generated and stored in `studio_invites` with `expires_at`
12. Token is embedded in an invite URL
13. New user follows the link, signs up, and is automatically added to the studio with the assigned role
14. Invite is marked as `accepted_at`; subsequent use of the same token is rejected

Phase 4 · Core Feature Pages

WHAT HAPPENED: All seven pages were migrated from the single-tenant PegBit OS into the multi-tenant Sydence architecture.

For each page, the migration process was:

15. Add `studio_id` to all relevant queries
16. Add RLS policy for the page's table
17. Add role-based permission checks to API routes
18. Add UI-level permission guards (hide buttons / disable inputs for viewers)
19. Add configurable vocabulary (read from `studio_vocabulary` table)
20. Test with multiple studio accounts

CONFIGURABLE VOCABULARY: All label text in Sydence that the studio can rename is read from the `studio_vocabulary` table at runtime. Defaults (Track 1 Flagship, Brief, In Development, etc.) are seeded on studio creation. Admins can change any label in Settings → Configure and the change takes effect immediately across the entire workspace.

Phase 5 · Omos AI Integration

WHAT HAPPENED: The Omos AI system was built in layers, from basic chat to full action execution.

Layer 1 — Document Ingestion Pipeline

Built the upload → parse → chunk → embed → store pipeline:

21. POST /api/omos/ingest receives the file
22. pdf-parse (for PDFs) or mammoth (for DOCX) extracts raw text
23. Text is split into sections (heading + content pairs)
24. Each section sent to OpenAI text-embedding-3-small → 1536-dim vector
25. Section stored in studio_knowledge with studio_id, document_name, content, embedding

Layer 2 — Streaming Chat (RAG)

Built the question → search → answer → stream pipeline:

26. POST /api/studio-mind receives user message
27. Message is embedded (same model as ingestion)
28. pgvector cosine similarity search finds top-k chunks scoped to studio_id
29. Top chunks injected as context into the GPT-4 prompt
30. Response streamed to client via Server-Sent Events (SSE)
31. Message and response saved to os_omos_messages for audit trail

Layer 3 — Report Generation

Built four report types: Project Status, Team Digest, Sprint Retrospective, Stakeholder Deck. Each queries live studio data (projects, standups, ideas) and generates a structured markdown report.

Layer 4 — Action Execution

Built OpenAI function calling to detect action intent and execute write operations (log standup, update project status, submit idea). Actions are logged to os_omos_actions for the audit trail.

Phase 6 · Security Hardening

WHAT HAPPENED: A comprehensive security audit was performed across all API endpoints and pages. Two security issues were found and fixed:

Fix 1 — Missing Admin Check on Knowledge Base GET

Issue: GET /api/omos/documents (list all ingested documents) was missing an admin role check. Any authenticated studio member could list the knowledge base documents — not just admins.

Fix: Added the same role validation present in POST and DELETE — query studio_members for the user's role, verify it is owner or admin before returning data.

Fix 2 — Analytics View SECURITY DEFINER

Issue: The os_omos_daily_stats view was created with SECURITY DEFINER, meaning it ran as the view creator (bypassing RLS) rather than as the querying user. This could allow a member to see analytics data from any studio by querying the view directly.

Fix: Migration 004 — Recreated the view explicitly with SECURITY INVOKER (the default). Additionally created a dedicated os_omos_admin_stats table with its own RLS policy restricting access to admin roles only.

AUDIT RESULT: All 7 admin-only features now have multi-layered protection: UI-level (navigation hidden), API-level (403 returned), and database-level (RLS).

Phase 7 · Admin Features & Analytics

WHAT HAPPENED: The admin layer was completed with analytics, reporting, and studio management features.

Analytics Dashboard

Admins can view: daily questions asked via Omos, total answers returned, unique users who asked questions, total tokens consumed (in and out), and feedback ratings. Data is displayed in a time-series chart with a date-range selector.

Knowledge Base Management

Admins can: upload documents (PDF, DOCX, TXT), view all ingested documents with section counts, and delete documents (which removes all chunks from studio_knowledge). Processing status shows whether a document is being embedded or is ready for queries.

Vocabulary / Studio Configuration

Admins can rename all configurable labels in the studio: pipeline stage names, track names, kanban column names, role titles. Changes take effect immediately and are read at runtime from studio_vocabulary.

Member Management & Invites

Admins can: view all studio members and their roles, promote or demote members, create invite links for specific roles, revoke outstanding invites.

Phase 8 · Slack Integration

WHAT HAPPENED: Slack webhook integration was added for two notification types:

- Standup notifications: When a team member posts a standup, a formatted Slack message is sent to the configured channel.
- Weekly digest: A cron job generates a weekly summary report (via Omos) and sends it to Slack every Monday.

The webhook URL is stored per-studio in the settings table. The integration is optional — if no webhook is configured, notifications are silently skipped.

Phase 9 · Production Deployment

WHAT HAPPENED: Sydence was deployed to Vercel with automatic CI/CD from GitHub.

Environment	Branch	URL	Purpose
Development	Any feature branch	Preview URL (auto-generated by Vercel)	Feature testing, PR review
Staging	staging	Staging Vercel environment	Pre-production validation
Production	main	Production Vercel environment	Live platform

Environment variables (API keys, database URLs) are set in Vercel project settings — never in the codebase. The `.env.local` file is gitignored and must be configured locally from the project's secret store.

Phase 10 · Documentation & Testing

WHAT HAPPENED: Three documentation artefacts were produced:

- `OMOS_DOCUMENTATION.md` — Full Omos system documentation (2,261 lines) covering architecture, schema, API reference, security model, and maintenance.
- `SYDENCE_DOCUMENTATION.md` — Complete platform documentation (1,228 lines) covering all features, the database schema, the user journey, admin features, API, and deployment.
- `TESTING_HANDOUT.md` — 27-section QA checklist for systematic testing of every feature.
- `TESTER_GUIDE.md` — 14-scenario guided walkthrough for feature testing (this document's companion).

08 API Reference Summary

All API routes require a Bearer token in the Authorization header. Routes return JSON. SSE routes stream text/event-stream.

Method	Endpoint	Access	Purpose
POST	/api/studio-mind	Member+	Chat with Omos. Streams SSE response.
GET	/api/omos/analytics	Admin+	Fetch Omos usage analytics.
POST	/api/omos/ingest	Admin+	Upload and process a knowledge document.
GET	/api/omos/documents	Admin+	List all ingested knowledge documents.
DELETE	/api/omos/documents	Admin+	Remove a document and all its chunks.
POST	/api/omos/reports	Admin+	Generate a report (project status, digest, etc).
GET	/api/omos/reports	Admin+	List previously generated reports.
POST	/api/omos/feedback	Member+	Submit thumbs up/down on a message.
GET	/api/projects	Member+	List projects for the studio.
POST	/api/projects	Member+	Create a new project.
PATCH	/api/projects/[id]	Member+	Update project status / stage.
GET	/api/ideas	Member+	List ideas for the studio.
POST	/api/ideas	Member+	Submit a new idea.
PATCH	/api/ideas/[id]	Admin+	Approve or reject an idea.
GET	/api/standups	Member+	List standup entries.
POST	/api/standups	Member+	Post a new standup.
GET	/api/settings	Member+	Get studio settings.
PATCH	/api/settings	Admin+	Update studio settings.
GET	/api/settings/members	Admin+	List studio members.
PATCH	/api/settings/members	Admin+	Update a member's role.
POST	/api/settings/invites	Admin+	Create a new invite token.
DELETE	/api/settings/invites	Admin+	Revoke an invite.

09 The Road Ahead

12-Week Pilot Roadmap

Phase	Weeks	Build Focus	Pilot Focus	Grant Focus
Architecture	1–2	Repo fork. Supabase multi-tenant schema. RLS. Vocab structure locked.	Identify 10 candidate studios. 3 conversations. Listen, do not pitch.	Vercel, NVIDIA Inception, Supabase, AWS Activate, Microsoft.
SaaS Core	3–5	Studio creation. Member invites. Per-studio Omos. Stripe wiring (not live).	Confirm 3–5 pilots. Share Loom walkthrough. Agree start date.	Record Sydence + Omos demo. Apply AWS Activate (\$5K). Microsoft (\$150K).
Feature Migration	6–9	All 7 pages multi-tenant. Configurable vocabulary live. PostHog analytics.	Weekly feedback calls. Document every friction point and vocab choice.	Write founder story. Pitch deck (10 slides). Research investors.
Pilot Ready	10–12	Marketing site. Empty state copy. Feedback form inside OS.	Onboard pilots. Onboarding call per studio. Weekly check-ins.	Apply Google Startups Cloud (\$200K). Warm investor outreach. Demo with pilots.

Phase 2 Features (Post-Pilot)

Features to build after pilot validation, prioritised by pilot feedback:

Feature	Priority	Why
Stripe billing (live payments)	P1	Needed to convert pilots to paying customers. Stripe for global; Paystack for NGN.
Knowledge Health Score	P1	The full Omos intelligence engine. Gives studios a single score (0-100) for their knowledge quality.
Witness Mode (immutable logs)	P2	Every Omos answer written to immutable audit log. Legally defensible record for compliance and HR disputes.
African language support	P2	Phase 1: Yoruba and Hausa. A question asked in Yoruba gets a Yoruba answer with the same precision and sourcing.

Mobile enhancements	P2	Responsive web works on mobile now. Native app or PWA with push notifications is Phase 2.
Slack integration for Omos	P3	Ask Omos questions directly from Slack. Answers posted back to the channel.
Advanced analytics (PostHog)	P3	Feature usage heatmaps, DAU/MAU trends, Omos engagement funnel.
SSO (Enterprise)	P3	Single sign-on for studios that require it. Enterprise tier.
Sydence marketing site	P1	Required for public launch. Not for pilot onboarding.

10 Pricing Model

Per-seat pricing aligned with the dominant convention across all competitors. Enables ARR storytelling for investors and expansion revenue as studios grow.

Tier	Price	Target	Includes
Free	\$0	1–3 users	All 7 pages. Omos: 3 questions/user/day. 10 documents max. Bottom-of-funnel acquisition.
Studio	\$15 / seat / month	4–15 users	All 7 pages. Omos: 20 questions/user/day. Unlimited documents. Standard support.
Studio Pro	\$25 / seat / month	15–30 users	All above + advanced analytics, Knowledge Health Score, priority support.
Enterprise	Custom from \$35 / seat	30+ users	Custom, SSO, white-label Omos, dedicated success, SLA guarantee.
Africa	40–50% discount on all	All	\$8–12/seat/month in Naira via Paystack. No Western competitor can match this without destroying unit economics.

THE PITCH

Never start with "here is what Sydence costs." Start with "here is what your current stack costs — \$465/month for a 10-person team." Then show Sydence at \$150/month for the same team. That is a 68% cost reduction for a product that does more.

11 Competitive Position

Competitor	Their Position	Sydence Response
Notion	Flexible wiki and light PM. \$10–20/user/mo.	Notion makes you build a studio OS. Sydence is one.
Linear	Opinionated PM for engineers. \$11/user/mo.	Linear is for engineers shipping tickets. Sydence is for studios shipping work.
Asana / Monday	Generic task and project management. \$10–25/user/mo.	They manage tasks. Sydence runs the studio.
Workamajig	Agency management. \$40–60/user/mo. No AI.	Built for ad agencies in 2005. Sydence is built for studios in 2026.
Glean	RAG AI. \$50/user/mo. 100-seat floor. Fortune 500.	Glean is RAG for the Fortune 500. Omos is RAG for the studio long tail.
Geekbot / Range	Standalone standup bots. \$2.50–8/user/mo.	Standups should not be a separate \$8/seat tool. Ours is built in.
Height (shutdown)	Horizontal AI PM. Shut down September 2025.	Height was horizontal. Sydence is vertical. That is why Height failed.

The Four Moats

Differentiator	Why no one else has it
Opinionated architecture, configurable vocabulary	The workflow loop is fixed. Studios rename the stages and tracks. Switching cost compounds with every label renamed. Notion is a canvas. Asana is generic. Nobody has built the opinionated studio workflow.
Omos — studio-specific AI	RAG assistant trained on the studio's own uploaded documents. Answers cite source and section. Never hallucinates. Glean does this but charges \$50/user with a 100-seat floor. No studio tool includes it. Omos is embedded.
Progression layer	Built-in talent development: levels, badges, weekly goals, connected directly to project delivery inside the OS. No PM tool has this. Lattice does HR reviews at \$15/user extra but is not connected to real project work.
Colony Lab	R&D experiments are first-class objects inside the OS. Studios document what they try, not just what they ship. No competitor treats organisational learning as a product feature. This category does not exist yet.

12 Developer Handover Guide

This section is for developers taking over the Sydence codebase. It covers how to run the project locally, the key files and directories, database setup, and how to deploy changes.

Prerequisites

- Node.js 18+
- npm or yarn
- A Supabase project (free tier is fine for development)
- An OpenAI API key (for Omos embeddings and chat)
- Optional: A Slack App with an Incoming Webhook URL

Step-by-Step Local Setup

32. Clone the repository from GitHub
33. Run npm install in the project root
34. Copy .env.example to .env.local and fill in the values (see table below)
35. Run the migrations against your Supabase project (SQL editor or Supabase CLI)
36. Run npm run dev
37. Open <http://localhost:3000> and sign up

Required Environment Variables

Variable	Where to get it	Purpose
NEXT_PUBLIC_SUPABASE_URL	Supabase Dashboard → Settings → API	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase Dashboard → Settings → API	Client-side anonymous key (safe to expose)
SUPABASE_SERVICE_ROLE_KEY	Supabase Dashboard → Settings → API	Server-side admin key — NEVER expose to client
OPENAI_API_KEY	platform.openai.com → API Keys	Used for embeddings and Omos chat completions

SLACK_WEBHOOK_URL (optional)	api.slack.com → Apps → Incoming Webhooks	Per-studio webhooks are stored in DB; this is the fallback
------------------------------	--	--

Key Directories

Path	What lives here
app/	Next.js App Router pages and layouts
app/api/	All API routes (one folder per endpoint)
app/api/studio-mind/	Omos chat endpoint with SSE streaming
app/api/omos/	All Omos-specific routes (ingest, documents, reports, analytics, feedback)
app/api/settings/	Studio settings, members, invites
components/	Shared React components
contexts/	React contexts (useAuth, useStudio)
migrations/	Numbered SQL migration files (run in Supabase)
scripts/	Build and generation scripts
OMOS_DOCUMENTATION.md	Full Omos system documentation
SYDENCE_DOCUMENTATION.md	Full Sydence platform documentation
TESTING_HANDOUT.md	QA test checklist
TESTER_GUIDE.md	Guided scenario-based test walkthrough

Database Migrations

Run each migration file in numbered order against your Supabase project:

38. Open Supabase Dashboard → SQL Editor
39. Paste the content of migrations/001_initial_schema.sql and run
40. Continue with 002, 003, 004 in order
41. Each migration is idempotent (uses CREATE TABLE IF NOT EXISTS and DROP POLICY IF EXISTS)

Do not skip migrations. Each one depends on tables or policies created by the previous.

Deployment

42. Push code to the main branch on GitHub

43. Vercel auto-detects the push and starts a deployment
44. Environment variables are read from Vercel project settings (not the code)
45. Deployment typically takes 60–90 seconds
46. Use Vercel → Deployments to roll back if needed (click Re-deploy on any previous deployment)

When Adding a New Feature

47. Write a migration for any new table (migrations/005_your_feature.sql)
48. Add RLS policies to the migration for every new table
49. Add the studio_id column to every table
50. Validate the API route with a Bearer token test before wiring the UI
51. Add role-based checks to the API route before any data operation
52. Add UI-level permission guards (hidden buttons, disabled inputs for viewers)
53. Update SYDENCE_DOCUMENTATION.md or OMOS_DOCUMENTATION.md with the new feature
54. Add a test scenario to TESTER_GUIDE.md

13 Risks & Mitigations

Risk	Severity	Mitigation
Studios not switching from Notion	High	Switching cost is psychological (sunk cost). Offer a 90-minute onboarding call. Make switching feel effortless and personally supported.
Notion / ClickUp AI commoditisation	Medium	Generic AI features cannot replicate Omos. The playbook, IP policy, and progression rules are not in Notion AI. They are in Omos.
API cost control at scale	Medium	The 5-question/day cap on free tier controls costs. Paid caps (20–30 q) are priced with margin. At ~\$0.01–0.02/question, 30 q/user/day costs under \$1/user/month.
Africa-only perception	Low	"Designed for the global studio long tail, with African pricing as the regional wedge." Get one pilot outside Nigeria in the first cohort.
Capacity risk — one person building two products	Critical	Most significant operational risk. 12-week timeline is realistic for a dedicated developer. If split between Sydence and ExamSurf simultaneously, one will slip. This must be explicitly planned.

14 Grant Strategy

Gate 1 — Apply This Week (No External Users Required)

Programme	Value	Key Requirement
Vercel Free Tier	Free forever	GitHub account and codebase deployed on Vercel. No form required.
NVIDIA Inception	AI compute	20-minute form. AI use case: Omos as proprietary AI infrastructure.
Supabase for Startups	Up to \$2,500	Already using Supabase. Describe multi-tenant RLS and pgvector.
AWS Activate (Founders)	\$5,000 + \$300	30-minute form. Product description + industry + team size.
Vercel Startups Programme	Credits (varies)	Short form. Demo link or live URL.
Microsoft Founders Hub	Up to \$150,000	LinkedIn + custom email + demo video link + product description.

Gate 2 — After 3 Pilot Studios

Programme	Value	Trigger Condition
Google for Startups Cloud	Up to \$200,000	Live product + 3 pilot studios OR 30 ExamSurf students.
AWS Activate (Portfolio)	\$25K–\$100K	Requires accelerator membership.

Accelerator Track (June / July 2026)

VERCEL AI ACCELERATOR

\$8M+ pool. 40 teams selected globally. Lead angle: Omos is a purpose-built AI infrastructure layer powering two products across two markets (B2B studio management + B2C education). This is stronger than a single AI feature.

SYDENCE

The operating system for studios that build things.

Powered by Omos · Built by PegBit Studio · Lagos, Nigeria
v2.0 · Production Ready · May 2026